

# Cetus CLMM

## Security Assessment

June 6th, 2025 — Prepared by OtterSec


Robert Chen r@osec.io

Michał Bochnak embe221ed@osec.io

Sangsoo Kang sangsoo@osec.io

# Table of Contents


**Executive Summary** **2**

Overview 2

Key Findings 2

Scope 2

**Findings** **3**

**Vulnerabilities** **4**

OS-CLM-ADV-00 | Role Removal 5

OS-CLM-ADV-01 | Possible Underflows 6

**General Findings** **7**

OS-CLM-SUG-00 | Incorrect Sqrt Price Boundaries 8

OS-CLM-SUG-01 | Incorrect Rounding Down 9

OS-CLM-SUG-02 | Invalid Pool Creation 10

OS-CLM-SUG-03 | Start Not Included 11

OS-CLM-SUG-04 | Pool Creation with Insufficient Liquidity 12

OS-CLM-SUG-05 | Lack of Handling of Malicious Positions 13

OS-CLM-SUG-06 | Missing Validation Logic 14

OS-CLM-SUG-07 | Code Maturity 16


## Appendices


**Vulnerability Rating Scale** **18**

**Procedure** **19**


© 2025 Otter Audits LLC. All Rights Reserved.

# 01 — Executive Summary

## Overview

Cetus Protocol engaged OtterSec to assess the `cetus-clmm-sui` program. This assessment was conducted between November 15th and November 30th, 2024. We conducted a follow-up review from May 27th to June 6th, 2025. For more information on our auditing methodology, refer to Appendix B.

## Key Findings

We produced 10 findings throughout this audit engagement.

We made a recommendation to modify the codebase for enhanced functionality and efficiency (OS-CLM-SUG-07) and suggested ensuring the pool is supplied with sufficient liquidity prior to creation (OS-CLM-SUG-04). We also advised implementing proper validation to mitigate potential security issues (OS-CLM-SUG-06).

## Scope

The source code was delivered to us in a Git repository at https://github.com/CetusProtocol/cetus-clmm-sui. This audit was performed against e3564d4. We further reviewed the changes in PR#1 and PR#2, and 3eab21e in cetus compensation repo.

**A brief description of the program is as follows:**

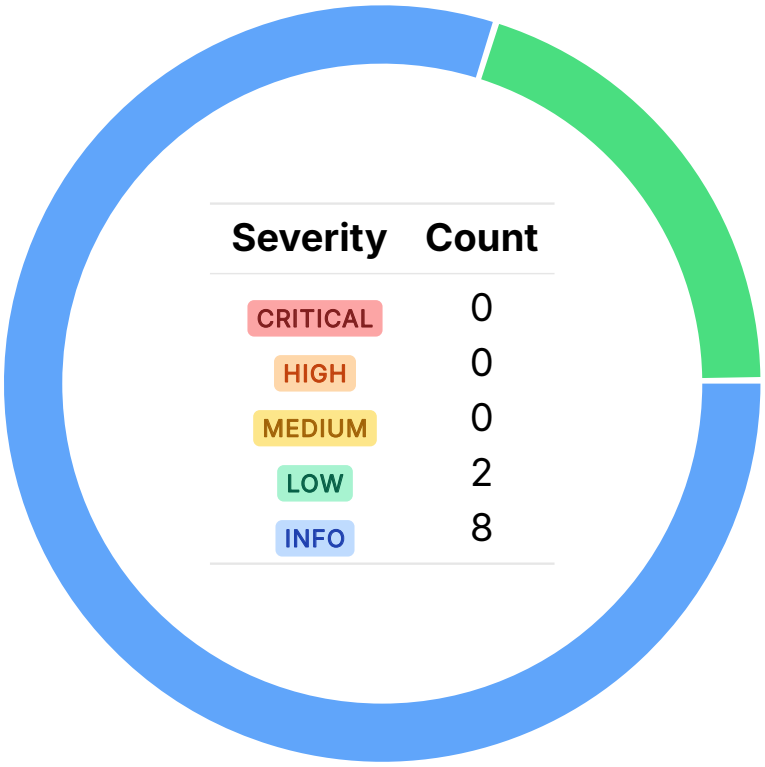
| Name | Description |
| --- | --- |
| cetus-clmm-sui | The Cetus CLMM on the Sui blockchain optimizes decentralized exchange functionality by allowing liquidity providers to allocate their assets within specified price ranges, improving capital efficiency and yield opportunities. |

© 2025 Otter Audits LLC. All Rights Reserved.

# 02 — Findings

Overall, we reported 10 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



| Severity | Count |
| --- | --- |
| CRITICAL | 0 |
| HIGH | 0 |
| MEDIUM | 0 |
| LOW | 2 |
| INFO | 8 |

© 2025 Otter Audits LLC. All Rights Reserved.

# 03 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

| ID | Severity | Status | Description |
|---|---|---|---|
| OS-CLM-ADV-00 | LOW | RESOLVED | Removal of non-existent role may affect other roles in `acl::remove_role` |
| OS-CLM-ADV-01 | LOW | RESOLVED | In `get_next_sqrt_price_{a_up,b_down}` the underflow may happen |

© 2025 Otter Audits LLC. All Rights Reserved.

## Role Removal LOW OS-CLM-ADV-00

### Description

The subtraction operation in `acl::remove_role` can affect lower bits if the specified role does not exist.

>_ *cetus_clmm/sources/acl.move* RUST

```
public fun remove_role(acl: &mut ACL, member: address, role: u8) {
    assert!(role < 128, ERoleNumberTooLarge);
    if (linked_table::contains(&acl.permissions, member)) {
        let perms = linked_table::borrow_mut(&mut acl.permissions, member);
        *perms = *perms - (1 << role);
    }
}
```

### Remediation

Make sure that `perms` has the `role` enabled before removing it.

### Patch

Resolved in 002ca7c.

© 2025 Otter Audits LLC. All Rights Reserved.

## Possible Underflows LOW OS-CLM-ADV-01

### Description

It is possible that the underflow will happen in `get_next_sqrt_price_{a_up,b_down}`.

>_ *cetus_clmm/sources/math/clmm_math.move* RUST

```rust
let new_sqrt_price = if (by_amount_input) {
    (math_u256::div_round(numberator, (liquidity_shl_64 + product), true) as u128)
} else {
    (math_u256::div_round(numberator, (liquidity_shl_64 - product), true) as u128)
};
```

>_ *cetus_clmm/sources/math/clmm_math.move* RUST

```rust
let new_sqrt_price = if (by_amount_input) {
    sqrt_price + delta_sqrt_price
} else {
    sqrt_price - delta_sqrt_price
};
```

### Remediation

Ensure that the subtraction will not abort

### Patch

Resolved in 002ca7c.

© 2025 Otter Audits LLC. All Rights Reserved.

# 04 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

| ID | Description |
|---|---|
| OS-CLM-SUG-00 | `flash_swap_internal` allows `sqrt_price_limit` to be equal to `{max,min}_sqrt_price` |
| OS-CLM-SUG-01 | Shift left operation rounds down the result, however the function may be called with `round_up = true` |
| OS-CLM-SUG-02 | `create_pool_internal` allows setting `initialize_price` to an incorrect value |
| OS-CLM-SUG-03 | `start` value is not included in results of `fetch_{ticks,positions}` functions |
| OS-CLM-SUG-04 | Users are allowed to create a pool by providing full-range liquidity but the function fails to validate the amounts of `coin_a` and `coin_b`. This oversight enables users to supply negligible amounts of one coin resulting in insufficient pool liquidity. |
| OS-CLM-SUG-05 | It is recommended to split `emergency_restore_pool_state` into two steps, by first removing any malicious liquidity positions, and then performing the price-restoring swap. |
| OS-CLM-SUG-06 | There are several instances where proper validation is not performed, resulting in potential security issues. |
| OS-CLM-SUG-07 | Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices. |

© 2025 Otter Audits LLC. All Rights Reserved.

## Incorrect Sqrt Price Boundaries

OS-CLM-SUG-00

### Description

In `flash_swap_internal` it is possible to set `sqrt_price_limit` to `{max,min}_sqrt_price`. Therefore, after the swap, the pool may be left in an incorrect state.

*>_ cetus_clmm/sources/pool.move* RUST

```
if (a2b) {
    assert!(
        pool.current_sqrt_price > sqrt_price_limit && sqrt_price_limit >= min_sqrt_price(),
        EWrongSqrtPriceLimit
    );
} else {
    assert!(
        pool.current_sqrt_price < sqrt_price_limit && sqrt_price_limit <= max_sqrt_price(),
        EWrongSqrtPriceLimit
    );
};
```

### Remediation

Replace `>=` and `<=` operators with `>` and `<`.

### Patch

Resolved in 0a53124.

© 2025 Otter Audits LLC. All Rights Reserved.

## Incorrect Rounding Down OS-CLM-SUG-01

### Description

In `get_liquidity_from_a` the `round_up` parameter is ignored during the `numberator` calculation.

*>_ cetus_clmm/sources/math/clmm_math.move* RUST

```
let numberator = (full_math_u128::full_mul(sqrt_price_0, sqrt_price_1) >> 64) * (amount_a as
    ↪  u256);
```

### Remediation

Do not round down the result if `round_up` is set to `true`

### Patch

Resolved in PR#28.

© 2025 Otter Audits LLC. All Rights Reserved.

## Invalid Pool Creation OS-CLM-SUG-02

### Description

In `create_pool_internal`, it is possible to set `initialize_price` to `max_sqrt_price`, which should be unreachable.

>_ *cetus_clmm/sources/factory.move* RUST

```
assert!(
    initialize_price >= tick_math::min_sqrt_price() && initialize_price <=
        ↪  tick_math::max_sqrt_price(),
    EInvalidSqrtPrice
);
```

### Remediation

Replace `<=` operator with `<`.

### Patch

Resolved in 002ca7c.

© 2025 Otter Audits LLC. All Rights Reserved.

## Start Not Included

OS-CLM-SUG-03

### Description

The `start` value is not included in the results list returned from `fetch_{ticks,positions}`.

>_ *cetus_clmm/sources/tick.move* RUST

```rust
let opt_next_score = if (vector::is_empty(&start)) {
    skip_list::head(&manager.ticks)
} else {
    let score = tick_score(i32::from_u32(*vector::borrow(&start, 0)));
    skip_list::find_next(&manager.ticks, score, false)
};
```

### Remediation

Include `start` value in the result.

### Patch

Resolved in PR#28.

© 2025 Otter Audits LLC. All Rights Reserved.

## Pool Creation with Insufficient Liquidity OS-CLM-SUG-04

### Description

In the current implementation of `create_pool_v2_by_creation_cap` within `pool_create`, users have to provide full-range liquidity when creating a new pool. However, the function extracts `amount_a` and `amount_b` from the provided `coin_a` and `coin_b` without validating whether the amounts of `coin_a` and `coin_b` provided by the user are sufficient to contribute meaningful liquidity to the newly created pool.

*>_ cetus_clmm/sources/pool_creator.move* RUST

```
public fun create_pool_v2_by_creation_cap<CoinTypeA, CoinTypeB>(
    [...]
): Position {
    let (tick_lower_idx, tick_upper_idx) = full_range_tick_range(tick_spacing);
    let (amount_a, amount_b) = (coin::value(&coin_a), coin::value(&coin_b));
    let (position, coin_a, coin_b) =
        ↪  factory::create_pool_v2_by_creation_cap_<CoinTypeA,CoinTypeB>(
       [...]
    );
    [...]
}
```

Thus, it is possible to pass a small amount of one coin that only provides negligible liquidity to the pool, enabling the pool creation process to be abused to flood the system with pools with insufficient liquidity.

### Remediation

Ensure the pool is supplied with sufficient liquidity prior to creation.

### Patch

Resolved in 8d16203.

© 2025 Otter Audits LLC. All Rights Reserved.

## Lack of Handling of Malicious Positions OS-CLM-SUG-05

### Description

While `pool::emergency_restore_pool_state` aims to fix the pool price, it may not fully address issues from multiple malicious positions within the pool. Simply restoring the price will leave underlying compromised tick data, affecting future operations. Therefore, a two-step approach is recommended, where the function first removes the malicious positions and liquidity and consequently proceeds with the swap in the second step to restore the desired state of the pool.

### Remediation

Modify `pool::emergency_restore_pool_state` with the two-step process mentioned above.

### Patch

Resolved in 9f9b3f9.

© 2025 Otter Audits LLC. All Rights Reserved.

## Missing Validation Logic

OS-CLM-SUG-06

### Description

1. `pool::governance_fund_injection` and `pool::emergency_restore_pool_state` should verify the pool's paused status before execution. Proceeding while the pool is active and trades are occurring may result in unpredictable side effects and data corruption, impacting ongoing swaps, price calculations, and liquidity provision.
2. Add a pool manager authorization check to `pool::governance_fund_injection` to ensure that only authorized pool managers are allowed to add funds to the pool.
3. When restoring pool state, consider increasing `pool.liquidity` in specific cases, because when the attacker called `remove_liquidity` while `lower_tick ≤ current_tick < upper_tick`, it reduced `pool.liquidity`. Therefore, `pool::emergency_restore_pool_state` should explicitly verify if `pool.current_tick_index` is in `[tick_lower;tick_upper)` range, and update the `pool.liquidity` if so.

*>_ cetus_clmm/sources/pool.move* RUST

```
public fun remove_liquidity<CoinTypeA, CoinTypeB>(
    config: &GlobalConfig,
    pool: &mut Pool<CoinTypeA, CoinTypeB>,
    position_nft: &mut Position,
    delta_liquidity: u128,
    clock: &Clock,
): (Balance<CoinTypeA>, Balance<CoinTypeB>) {
    [...]
    // 4. pool liquidity
    if (
        i32::lte(tick_lower, pool.current_tick_index) &&
            i32::lt(pool.current_tick_index, tick_upper)
    ) {
        pool.liquidity = pool.liquidity - delta_liquidity;
    };
    [...]
}
```

4. In `add_liquidity_fix_coin`, ensure that `liquidity` is greater than zero.
5. Validate the relationship between `Pool` and `Position` objects explicitly.

### Remediation

Incorporate the validations stated above.

© 2025 Otter Audits LLC. All Rights Reserved.

### Patch

1. Issue #1 resolved in 9f9b3f9.
2. Issue #2 resolved in 9f9b3f9.
3. Issue #4 resolved in 002ca7c.
4. Issue #5 resolved in 002ca7c.

© 2025 Otter Audits LLC. All Rights Reserved.

## Code Maturity OS-CLM-SUG-07

### Description

1. Validate the `tick_spacing` parameter in `factory::add_allowed_pair_config` to ensure that only valid and meaningful tick spacings are allowed to be added to the allowed pair configuration.
2. `pool:check_pool_manager_role` should indeed utilize `is_pool_manager` to avoid duplicating the role-checking logic.

>_ *cetus_clmm/sources/config.move* RUST

```
// Check member is pool manager
public fun is_pool_manager(config: &GlobalConfig, member: address): bool {
    acl::has_role(&config.acl, member, ACL_POOL_MANAGER)
}

/// Check member has pool manager role
public fun check_pool_manager_role(config: &GlobalConfig, member: address) {
    assert!(acl::has_role(&config.acl, member, ACL_POOL_MANAGER), ENoPoolManagerPemission)
}
```

3. In `factory`, store `type_name::get<Coin>` in a variable within the `add_allowed_pair_config` to improve code efficiency, avoiding multiple calls to `type_name::get<Coin>`
4. The amount parameter in `pool::emergency_restore_pool_state` is redundant as the swap should execute until it reaches `target_sqrt_price`.
5. In `clmm_vester`, the generic type parameter `<T>` in `ClmmVester<T>` should be hardcoded to a Cetus coin.
6. In `emergency_restore_pool_state`, it is theoretically possible that no liquidity exists at the `target_sqrt_price`, resulting in the function to fail to reach the target and ultimately abort. Ensure proper error handling.

### Remediation

Implement the above-mentioned suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

### Patch

1. Issue #1 resolved in 8d16203.
2. Issue #2 resolved in 8d16203.
3. Issue #3 resolved in 8d16203.
4. Issue #4 resolved in 9f9b3f9.
5. Issue #5 resolved in 177beb6.

© 2025 Otter Audits LLC. All Rights Reserved.

# A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

**CRITICAL**

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

**HIGH**

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

**MEDIUM**

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

**LOW**

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

**INFO**

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

# B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.